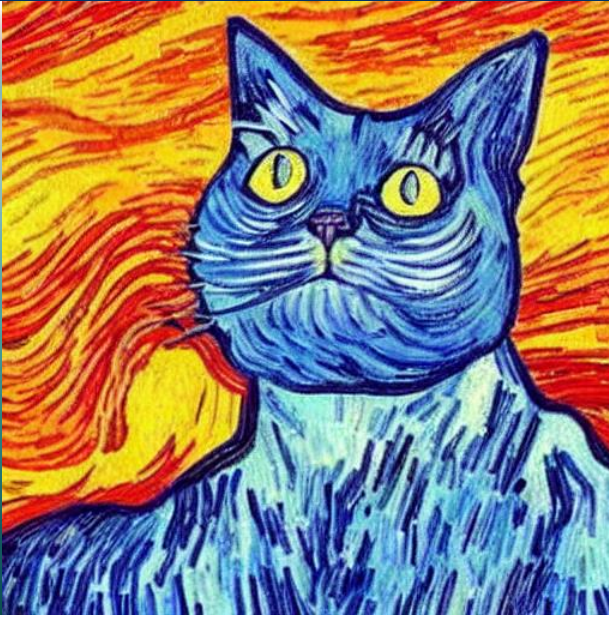
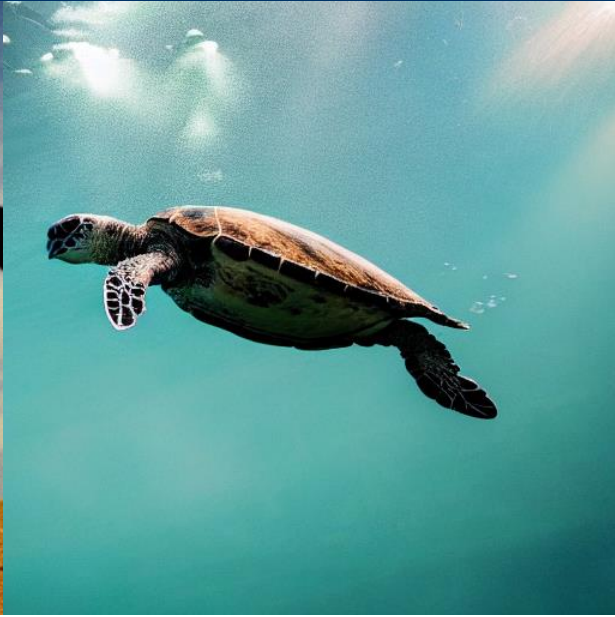
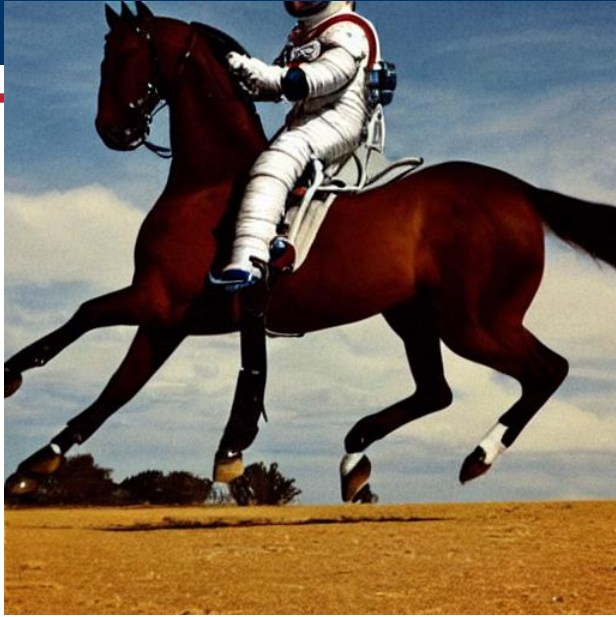




Generative Models for Computer Vision

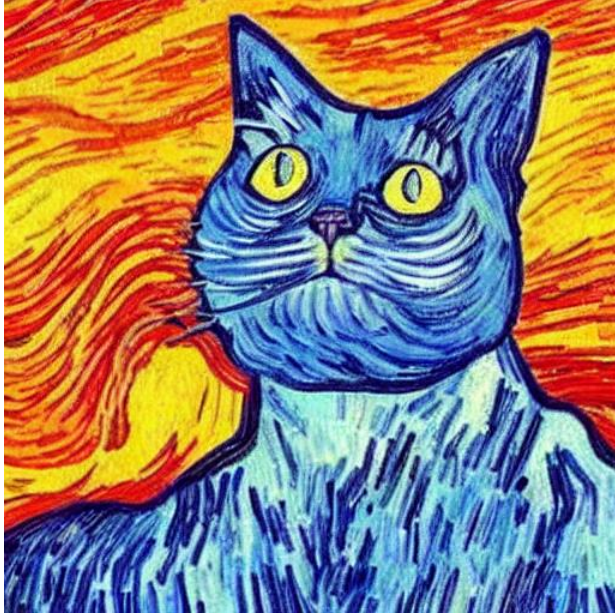
Ch7. Stable Diffusion

ONE LOVE. ONE FUTURE.



Có gì cần chú ý với những bức hình này?
→ Đều được sinh ra bởi stable diffusion

Tại sao chúng ta cần quan tâm?



"một chú mèo dễ
thương ở trong sa mạc
theo phong cách của
Van Gogh"

Mô hình của trí tưởng tượng

Có thể áp dụng trong nhiều lĩnh vực khác
nhau: hội họa, điện ảnh, y học...

Stable diffusion

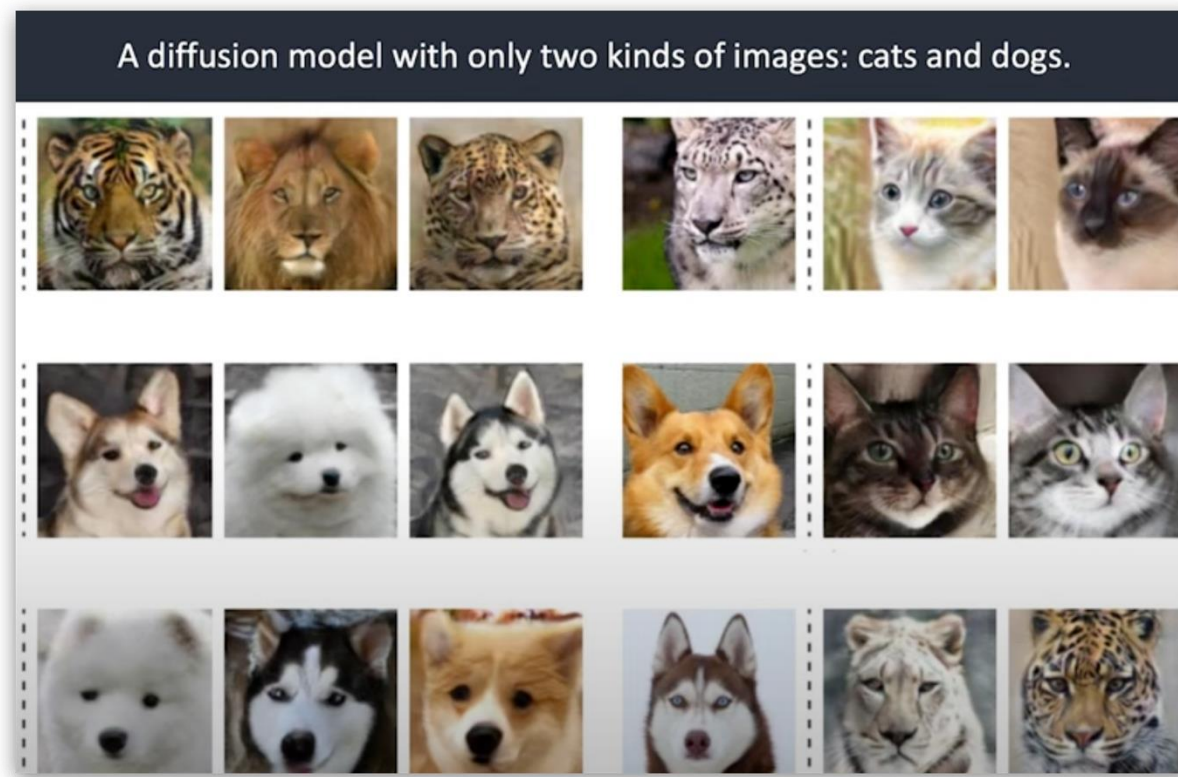
- Công cụ cho phép nhập vào dòng chữ (prompt) và sinh ra ảnh



<https://stable-diffusion-art.com/how-stable-diffusion-work>

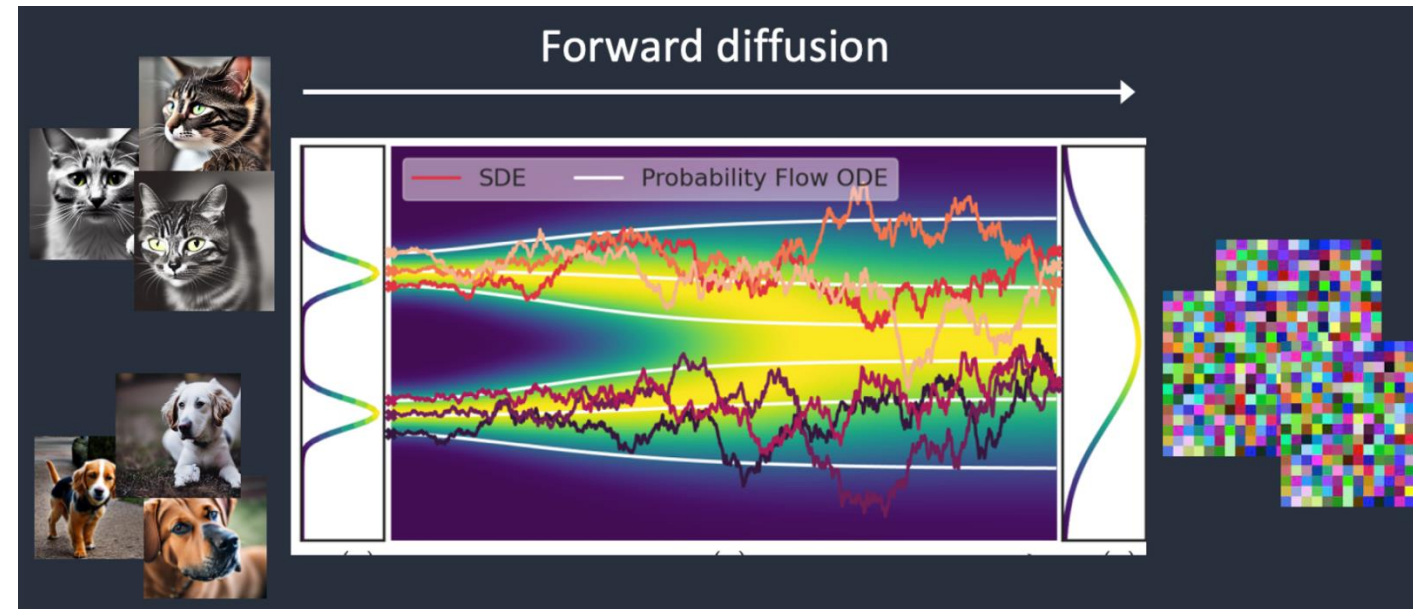
Diffusion model

- Mô hình khuếch tán (diffusion model) thuộc lớp bài toán học sâu
- Sinh ảnh giống với dữ liệu đã được học trong huấn luyện
 - Ví dụ học các ảnh về chó và mèo → sinh ra ảnh chó và mèo



Forward diffusion (1)

- Quá trình thêm nhiễu vào ảnh gốc (mèo và chó) từng bước để tạo ra ảnh nhiễu hoàn toàn
 - Mô hình học cách chuyển đổi từ dữ liệu có cấu trúc sang trạng thái nhiễu hoàn toàn
- ODE (Ordinary Differential Equation) - Phương trình vi phân thường
 - Mô tả sự thay đổi mà không có yếu tố ngẫu nhiên
 - Ứng dụng: Mô tả quá trình biến đổi có thể dự đoán của ảnh, được xác định rõ ràng, trong điều kiện ổn định, dễ kiểm soát
- SDE (Stochastic Differential Equation) - Phương trình vi phân ngẫu nhiên
 - Thêm yếu tố ngẫu nhiên: Mô hình dưới dạng nhiễu trắng hoặc biến ngẫu nhiên
 - Ứng dụng: mô tả quá trình khuếch tán nơi mỗi bước của quá trình thêm một mức độ ngẫu nhiên nhất định → hệ thống không hoàn toàn dự đoán được



Forward diffusion chuyển ảnh sang trạng thái nhiễu hoàn toàn

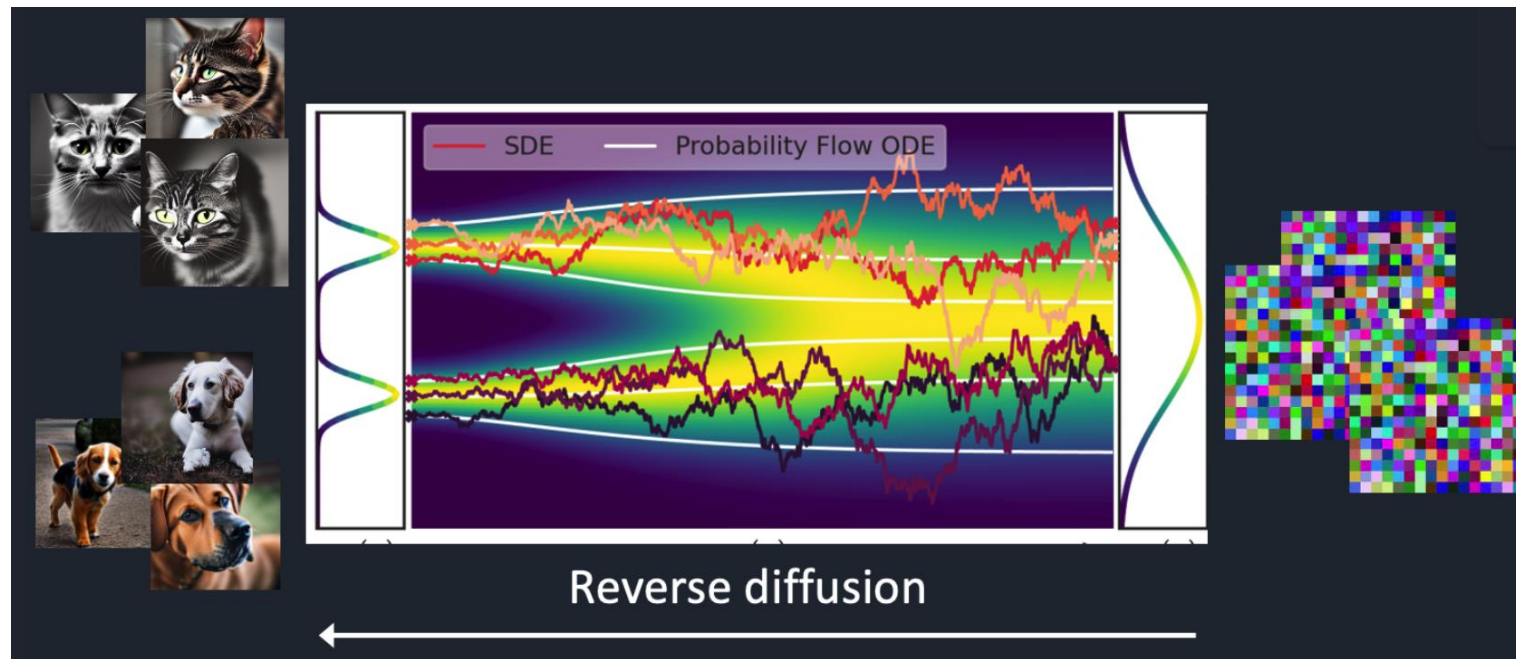
Forward diffusion (2)

- Quá trình thêm nhiễu ngẫu nhiên từng bước vào ảnh gốc
- Ảnh sau khi thêm nhiễu hoàn toàn không thể nhận diện được là chó hay mèo
- Giống như giọt mực nhỏ vào cốc nước: khi khuếch tán hoàn toàn, không biết giọt mực ở đâu nữa



Reverse diffusion

- Loại bỏ nhiễu từ ảnh ngẫu nhiên để tái tạo ảnh có cấu trúc (như chó hoặc mèo)
- Quá trình lặp lại nhiều bước để dần khôi phục thông tin
- Cơ chế hoạt động:
 - Mỗi bước, mô hình dự đoán lượng nhiễu trong ảnh và loại bỏ nhiễu này
→ ảnh thu được tương tự dữ liệu huấn luyện



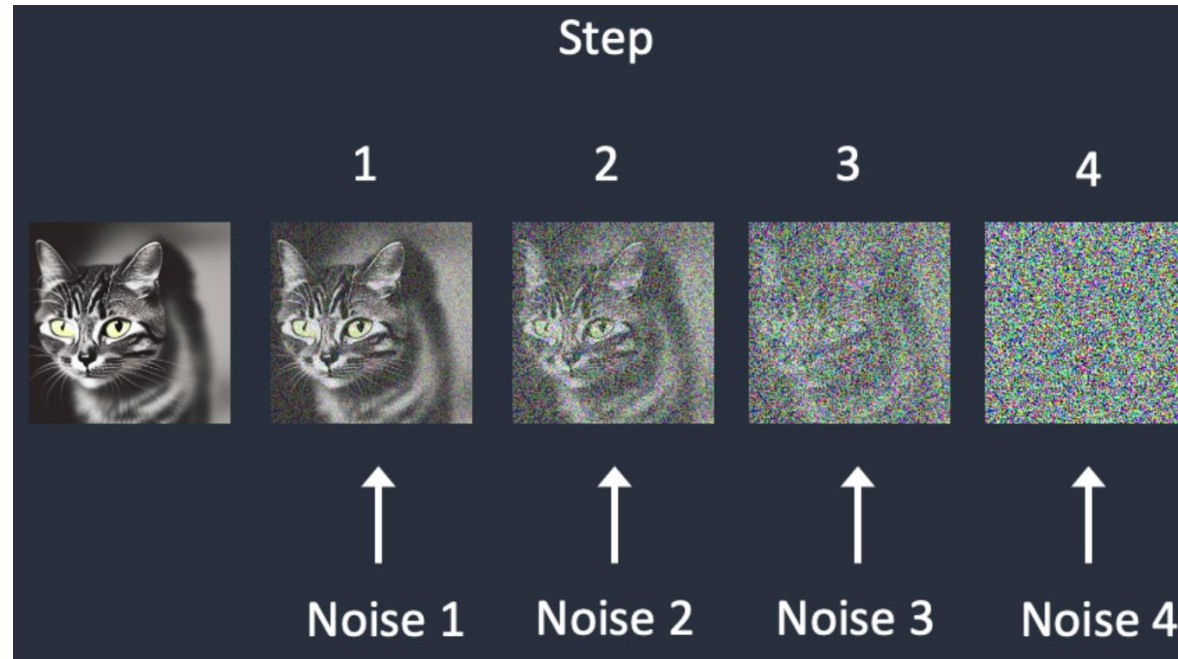
Reverse diffusion loại bỏ nhiễu từ ảnh ngẫu nhiên qua nhiều bước để tái tạo lại ảnh có ý nghĩa

Cách huấn luyện (1)

- Để mô hình có thể khôi phục ảnh từ nhiễu (reverse diffusion): cần phải biết bao nhiêu nhiễu đã thêm vào ảnh trong forward từ diffusion → Mục tiêu là mô hình có thể dự đoán được lượng nhiễu này
 - Nếu ta trừ đi lượng nhiễu đã được thêm vào, có thể sinh ra ảnh gốc (chó hay mèo)
- Các bước huấn luyện
 - Lấy ảnh huấn luyện: ảnh gốc (chó hay mèo)
 - Sinh nhiễu ngẫu nhiên
 - Thêm nhiễu vào ảnh gốc
 - Huấn luyện mô hình dự đoán lượng nhiễu
 - Đầu vào: ảnh bị nhiễu
 - Đầu ra: lượng nhiễu đã thêm vào (ground truth)
 - Điều chỉnh mô hình để giảm hàm loss → Tìm trọng số (weight) tối ưu
 - Mục tiêu: đưa ảnh vào thì mô hình có khả năng dự đoán được chính xác lượng nhiễu đã thêm vào

Cách huấn luyện (2)

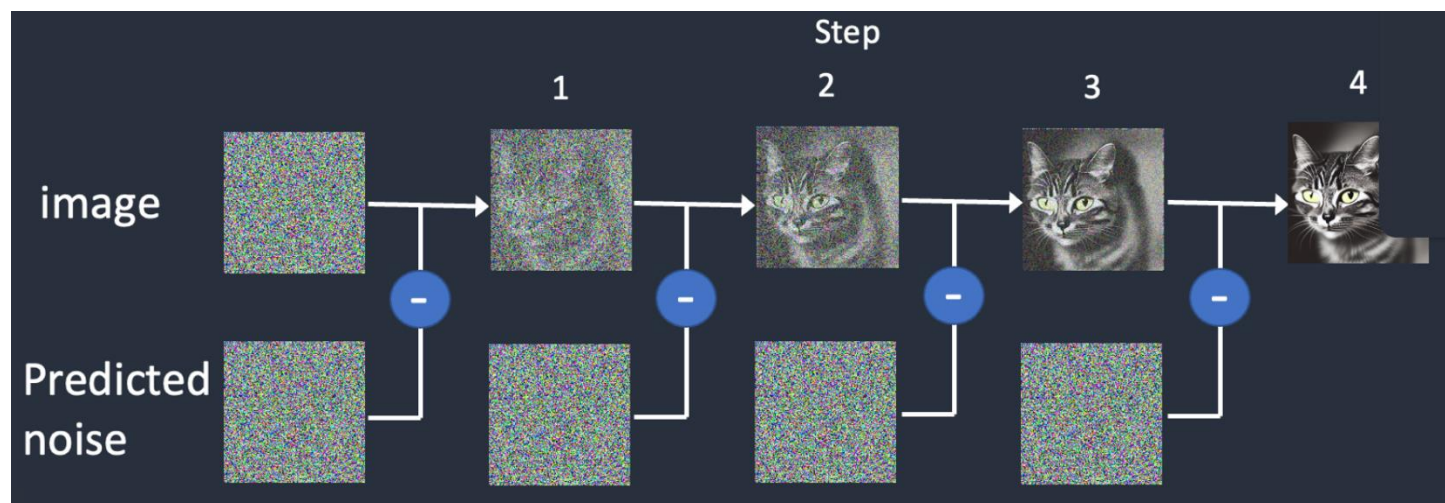
- Thêm nhiễu vào ảnh qua từng bước
- Sau khi huấn luyện, mô hình học được cách dự đoán lượng nhiễu đã thêm vào ảnh



Quá trình thêm nhiễu qua bốn bước làm ảnh trở nên nhiễu hoàn toàn, qua quá trình huấn luyện, mô hình có thể đoán được lượng nhiễu thêm vào

Sử dụng diffusion model để tái tạo ảnh

- Khởi đầu từ ảnh nhiễu hoàn toàn: không còn thông tin nhận diện
- Quá trình reverse diffusion:
 - Mô hình dự đoán lượng nhiễu cần loại bỏ tại mỗi bước
 - Loại bỏ dần lượng nhiễu dự đoán, thực hiện qua nhiều bước
- Kết quả cuối cùng: ảnh được tái tạo



Minh họa quá trình reverse diffusion: loại bỏ nhiễu từng bước để tái tạo ảnh

Vấn đề của diffusion model

- Không có điều kiện ràng buộc → khó kiểm soát nội dung đầu ra
- Tốc độ xử lý chậm vì cần xử lý từng điểm ảnh (pixel) qua nhiều bước

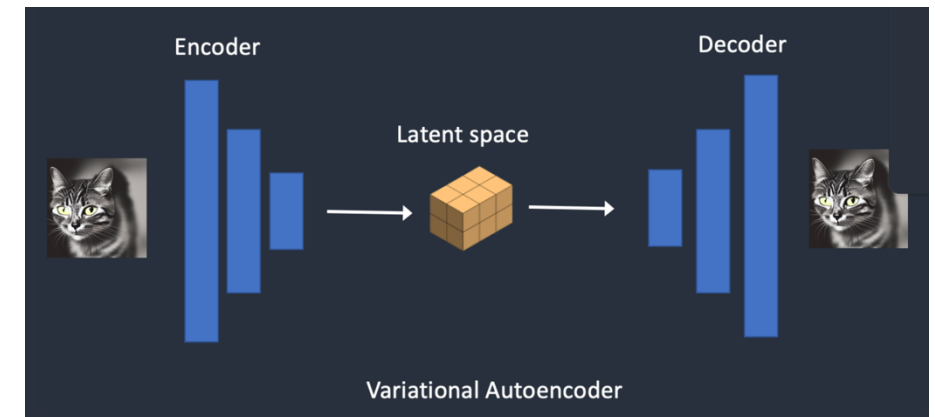


Ảnh minh họa không gian pixel: mỗi kênh màu có kích thước cụ thể và cần xử lý từng điểm ảnh

- Ảnh kích thước $512 \times 512 \times 3 \rightarrow 784432$ giá trị
- Nếu sử dụng 1000 bước $\rightarrow 784432 \times 1000 = 784432000$ phép tính $\rightarrow$ khối lượng tính toán khổng lồ

Stable diffusion (control speed)

- Latent space:
 - Giảm kích thước: VAE mã hóa ảnh gốc thành tensor nhỏ hơn (latent space), giữ lại các đặc trưng quan trọng
 - Nhanh hơn: Kích thước latent space chỉ $4 \times 64 \times 64 = 16384$ (nhỏ hơn 48 lần so với kích thước gốc $3 \times 512 \times 512$)
 - Tiết kiệm tài nguyên: xử lý trong latent space giảm yêu cầu về bộ nhớ và thời gian xử lý
 - Quy trình
 - Encode ảnh gốc sang latent space
 - Xử lý trong latent space (thêm/loại bỏ nhiễu)
 - Decode từ latent space thành ảnh đầu ra



VAE mã hóa và giải mã ảnh gốc, giúp xử lý nhanh hơn trong latent space

Tính toán trên stable diffusion model (1)

- Forward diffusion
 - Bắt đầu từ latent space (tensor)
 - Ảnh gốc được VAE Encoder nén thành tensor nhỏ hơn và có kích thước cố định, ví dụ [4, 64, 64]
 - Tensor này đại diện cho các đặc trưng quan trọng của ảnh nhưng kích thước nhỏ hơn nhiều so với ảnh gốc
 - Thêm nhiễu qua nhiều bước
 - Tại mỗi bước, một lượng nhiễu ngẫu nhiên được thêm vào tensor
 - Số lượng bước được thiết kế để tạo sự chuyển đổi mượt mà từ trạng thái có cấu trúc sang trạng thái hoàn toàn nhiễu

Tính toán trên stable diffusion model (2)

- Reverse diffusion

- Ý nghĩa:

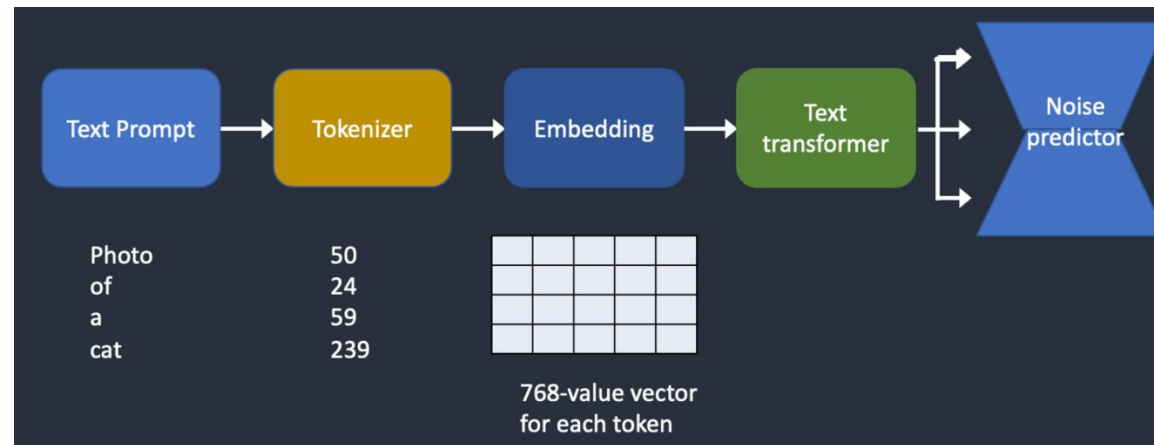
- Quá trình tái tạo lại thông tin từ trạng thái nhiễu trong latent space về trạng thái có cấu trúc
 - Đây là giai đoạn “khử nhiễu”: sử dụng các mô hình để dự đoán và loại bỏ nhiễu

- Quy trình:

- Bắt đầu từ latent space chứa nhiễu hoàn toàn:
 - Một tensor ngẫu nhiên, không có cấu trúc
 - Bộ dự đoán nhiễu (noise predictor):
 - Thường là một mô hình học sâu ví dụ U-Net để dự đoán lượng nhiễu đã được thêm vào tại mỗi bước
 - Sử dụng tensor hiện tại để xác định chính xác lượng nhiễu cần loại bỏ
 - Loại bỏ nhiễu:
 - Số lượng nhiễu dự đoán được trừ dần trong latent space (tensor) hiện tại
 - Quá trình này được thực hiện lặp lại qua một số bước nhất định để dần khôi phục thông tin
 - Giải mã bằng Decoder:
 - Sau khi reverse diffusion hoàn tất, latent space được giải mã thành ảnh đầu ra

Stable diffusion model (Control results)

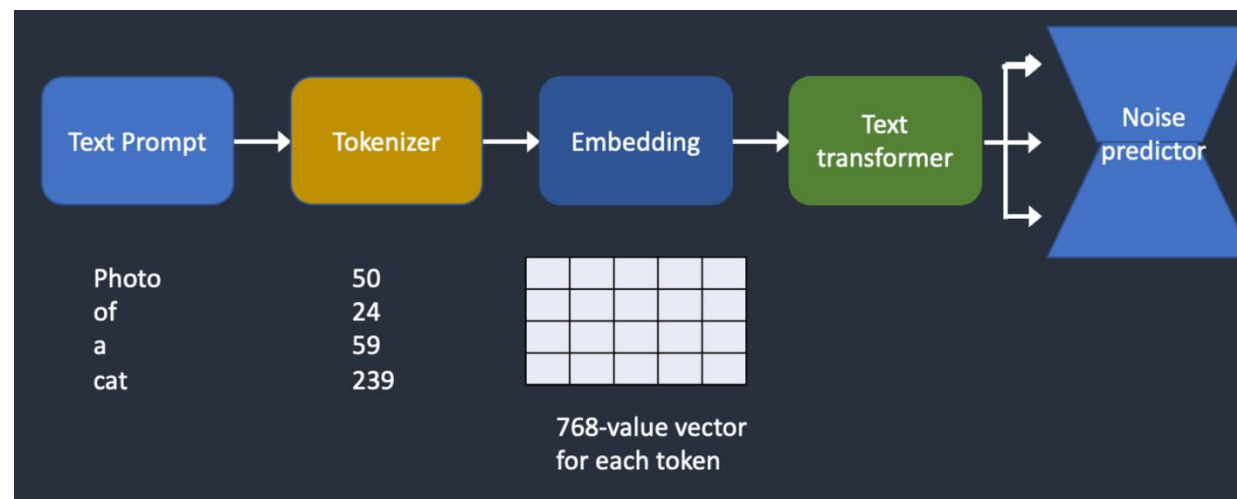
- Cho phép đưa vào prompt để điều khiển kết quả sinh ảnh
 - Text prompt: mô tả của người dùng
 - Tokenizer: phân tách prompt thành các token riêng biệt
 - Embedding: biểu diễn token dưới dạng vector nhiều chiều để mô tả ngữ nghĩa → tạo thành ma trận embedding
 - Text transformer: mô hình phân tích ngữ cảnh giữa các từ trong prompt để hiểu ý nghĩa tổng thể
 - Noise predictor: sử dụng thông tin từ prompt điều chỉnh quá trình reverse diffusion; đảm bảo lượng nhiễu được loại bỏ tại mỗi bước phản ánh đúng nội dung của prompt → đầu ra phù hợp với mô tả của người dùng



Quy trình xử lý prompt trong Stable diffusion từ text prompt đến noise predictor

Stable diffusion model (Control results)

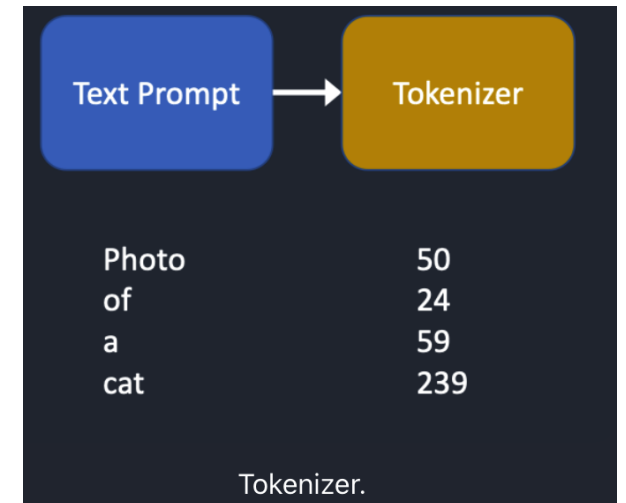
- Ý nghĩa:
 - Kiểm soát nội dung đầu ra: người dùng có thể định hướng rõ ràng nội dung và phong cách ảnh thông qua prompt
 - Linh hoạt và đa dạng: linh hoạt thay đổi, mô tả các nội dung phức tạp hơn để tạo ra ảnh sáng tạo
 - Tương tác trực quan: người dùng chỉ cần mô tả bằng ngôn ngữ tự nhiên, không cần hiểu rõ chi tiết bên trong



Quy trình xử lý prompt trong Stable diffusion từ text prompt đến noise predictor

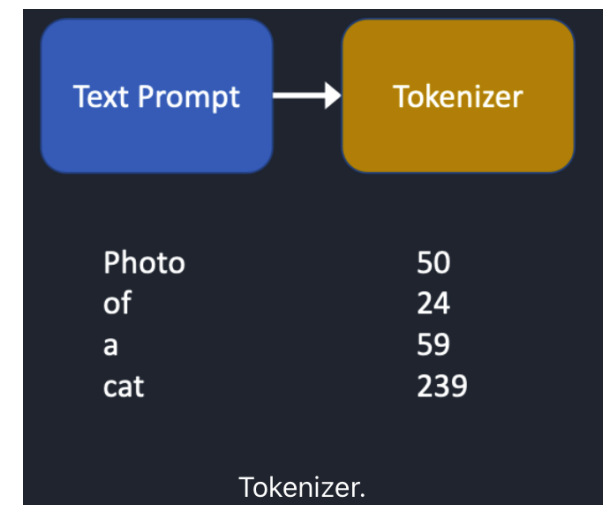
Tokenizer (1)

- Máy tính chỉ hiểu các con số:
 - Văn bản cần chuyển thành con số để xử lý (tokenization)
- Stable difusion v1 sử dụng CLIP Tokenizer (OpenAI):
 - Chuyển đổi văn bản thành token dựa trên mô hình đã học
- Không phải mỗi từ là 1 token
 - Tokenizer chỉ chuyển những từ đã học trong quá trình huấn luyện
 - Ví dụ: từ “dreambeach” chưa học sẽ được chia thành 2 token “dream” và “beach” đã được học



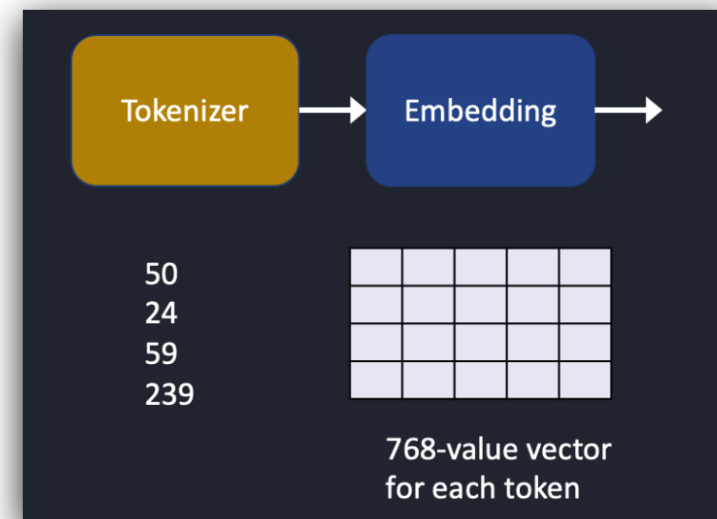
Tokenizer (2)

- Khoảng trắng cũng được tính:
 - Cụm từ “dream beach”: tạo ra hai token
 - “dream”
 - “[space]beach”
 - “dreambeach” tạo ra hai token
 - “dream”
 - “beach” (không có khoảng trắng trước “beach”)
- Giới hạn:
 - Mô hình stable diffusion thường có tối đa 77 token trong một prompt gồm 75 token nội dung và hai token đặc biệt <start> ,<end>
 - Không phải 77 từ vì một từ có thể nhiều token
 - Prompt dài hơn có thể bị cắt thành các phần (chunk) nhỏ hơn



Embedding

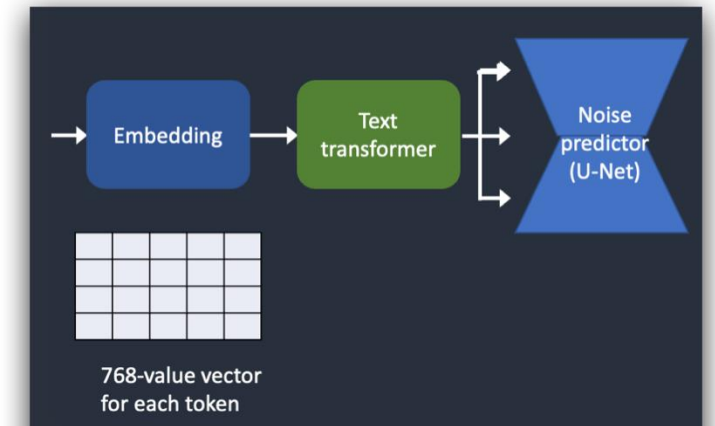
- Cách biểu diễn các token dưới dạng vector số nhiều chiều
- Trong stable diffusion v1,:
 - Mỗi token được ánh xạ thành một vector riêng biệt chứa 768 giá trị để biểu diễn ngữ nghĩa của token đó
 - Giá trị được học trong quá trình huấn luyện mô hình CLIP
- Lợi ích
 - Mô hình hiểu mối quan hệ giữa các từ với nhau → các từ ý nghĩa tương tự nhau có thể dùng thay thế cho nhau
 - man/Gentleman/guy
 - Monet/ Manet/ Degas (tên họa sĩ theo trường phái ấn tượng)
 - Điều chỉnh embedding có thể tạo ra kết quả độc đáo



Embedding ánh xạ mỗi token thành vector 768 giá trị

Text Transformer

- Vai trò
 - Xử lý đầu ra của embedding, chuẩn bị thông tin cho noise predictor
 - Tăng cường ngữ nghĩa và ngữ cảnh từ vector embedding
- Đặc điểm
 - Bộ chuyển đổi đa năng có thể xử lý nhiều loại dữ liệu
 - Nhãn lớp (class label): xử lý thông tin phân loại
 - Hình ảnh (image): hỗ trợ sinh ảnh
 - Bản đồ độ sâu (depth maps): sinh hình ảnh 3D
 - Kết hợp điều kiện (conditioning)
 - Tích hợp văn bản, hình ảnh, hoặc các thông tin bổ sung để tạo ra kết quả chính xác và phong phú hơn



Text transformer xử lý vector embedding từ văn bản

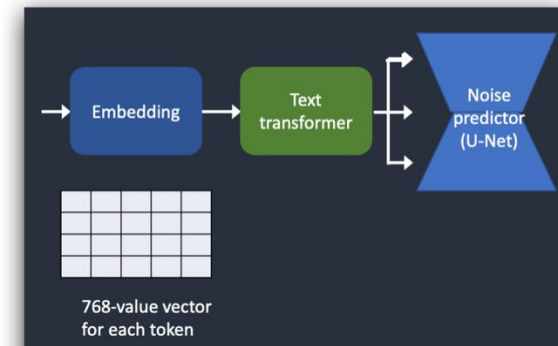
Cross-attention (1)

- Khái niệm

- Cơ chế giúp liên kết văn bản (text prompt) và quá trình xử lý ảnh trong Stable diffusion
- Được tích hợp trong Noise Predictor (ví dụ U-Net) đảm bảo thông tin prompt từ văn bản được áp dụng chính xác trong từng bước sinh ảnh

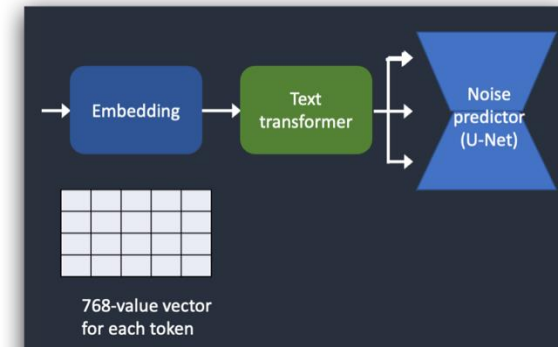
- Vai trò

- Liên kết giữa hình ảnh và văn bản:
 - kết nối embedding từ Text transformer với tensor nhiễu trong Noise Predictor
 - Giúp chuyển ngữ nghĩa từ prompt văn bản trong quá trình xử lý hình ảnh
- Hướng dẫn từng bước quá trình tạo ảnh
 - Trong mỗi bước giảm nhiễu (denoising), cross-attention liên tục tích hợp thông tin từ prompt, đảm bảo ảnh duy trì được các chi tiết quan trọng (màu sắc, đối tượng, bối cảnh)



Cross-attention (2)

- Prompt: “a man with blue eyes”
 - Embedding: các từ sẽ được chuyển thành các vector
 - Cross-attention: liên kết hai từ “blue” và “eyes, hướng dẫn Noise predictor thêm chi tiết “mắt xanh” trong hình ảnh
- Các kỹ thuật nâng cao:
 - Hypernetwork: Kỹ thuật này dùng cross-attention để chèn các phong cách mới vào ảnh sinh ra (tranh vẽ, hoạt hình)
 - LoRA Models: Tinh chỉnh trọng số của cross-attention để thay đổi phong cách hình ảnh hoặc cá nhân hóa hình ảnh



Các điều kiện khác (1)

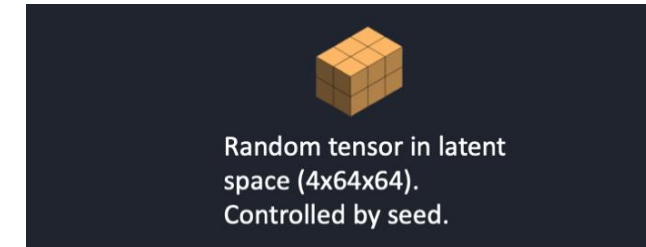
- Ảnh độ sâu (depth image): Mô hình depth-to-image sử dụng cả văn bản và hình ảnh độ sâu
 - Văn bản (Text prompt):
 - Cung cấp mô tả chi tiết về nội dung cần tạo.
 - Quy định các đối tượng và bối cảnh cần tạo
 - Ảnh độ sâu (Depth image):
 - Cung cấp thông tin về cấu trúc và khoảng cách trong không gian của đối tượng.
 - Đảm bảo ảnh sinh ra có cấu trúc không gian chính xác, tạo ra cảm giác thực tế về hình học và phối cảnh
 - Ví dụ
 - Prompt: “A mountain with a water fall”
 - Ảnh độ sâu: cho biết độ cao và khoảng cách các dãy núi và thác nước
 - Kết quả: Hình ảnh có phối cảnh 3D chính xác, thể hiện rõ chiều sâu giữa các đối tượng

Các điều kiện khác (2)

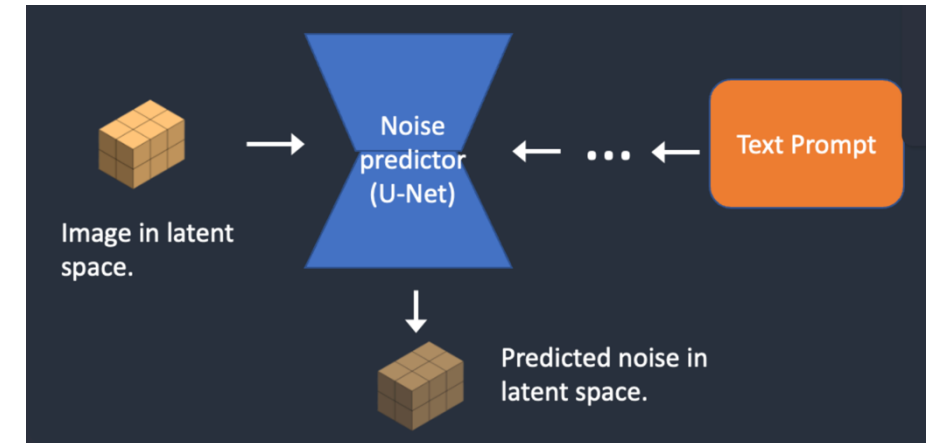
- ControlNet: sử dụng thêm các điều kiện khác vào stable diffusion để kiểm soát chi tiết đầu ra
 - Detected Outlines:
 - Sử dụng các đường viền của các đối tượng trong ảnh đầu vào được phát hiện để điều kiện hóa mô hình.
 - Ví dụ: Sinh ảnh dựa trên đường viền của một tòa nhà hoặc khuôn mặt
 - Ứng dụng: chuyển đổi ảnh phác thảo thành hình ảnh thực tế
 - Human Poses:
 - Sử dụng các tư thế của con người (pose estimation) để điều kiện hóa mô hình.
 - Ví dụ: một người đang chạy hoặc nhảy với động tác cụ thể
 - Ứng dụng: tạo ra hình ảnh nhân vật với tư thế chính xác dựa trên đầu vào
 - Các điều kiện khác
 - Keypoints: các điểm quan trọng như mắt, mũi, chân tay
 - Segment maps: phân vùng các phần của ảnh để tạo ra hình ảnh tương ứng

Text2image – Ý tưởng

- Đầu vào:
 - Prompt
 - Cung cấp bởi người dùng, mô tả nội dung cần sinh ra
 - Tensor khởi tạo ngẫu nhiên trong không gian ẩn latent space (ví dụ kích thước $4 \times 64 \times 64$)
 - Đại diện cho một ảnh chữ nhiễu, sẽ được giảm nhiễu (denoising) từng bước tạo ra ảnh cuối cùng
 - Seed điều khiển Tensor ngẫu nhiên này, cho phép tái tạo kết quả giống nhau khi chung seed
- Hoạt động:
 - Kết hợp tensor ngẫu nhiên và prompt
 - Tensor ngẫu nhiên và prompt được đưa vào Noise Predictor
 - Prompt hướng dẫn Noise Predictor điều chỉnh tensor để phù hợp với nội dung miêu tả
 - Noise predictor sử dụng cross-attention để kết hợp thông tin từ prompt và tensor
 - Quá trình được lặp lại qua nhiều bước cho đến khi tensor trở thành ảnh cuối cùng phù hợp với prompt



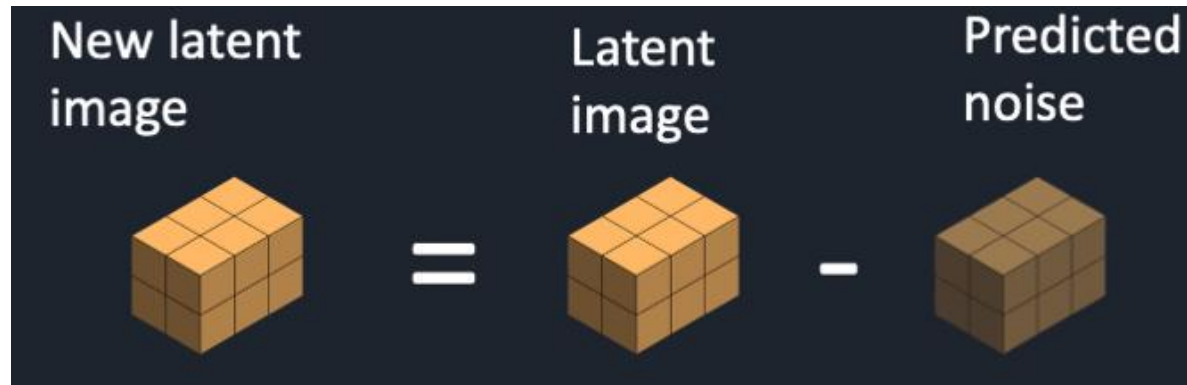
Tensor ngẫu nhiên được sinh ra trong latent space



Hoạt động của Text2image

Text2image – Quy trình lặp giảm nhiễu

- Lấy tensor ngẫu nhiên:
 - Tensor khởi tạo ngẫu nhiên được sinh ra trong không gian ẩn (latent space)
 - Tensor này đại diện cho hình ảnh ban đầu chứaa toàn bộ nhiễu
- Quá trình giảm nhiễu
 - Noise predictor được sử dụng để ước lượng nhiễu trong tensor hiện tại
 - Tensor được thay đổi bằng cách trừ đi lượng nhiễu đã được dự đoán trong tensor hiện tại
- Số vòng lặp
 - Có thể từ 15-20 lần hoặc nhiều hơn, mỗi lần làm giảm dần nhiễu trong tensor
 - Mỗi bước lặp giúp tensor tiệm cận đến trạng thái ít nhiễu hơn → ảnh hoàn chỉnh



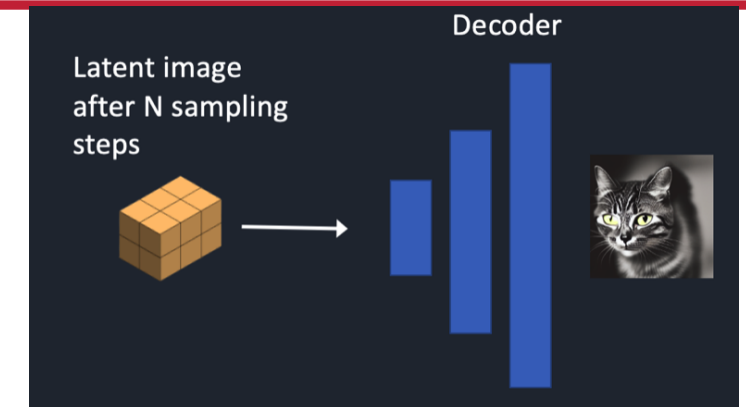
Tensor được xử lý trong mỗi vòng lặp để giảm nhiễu

x 15-20 times

Quy trình được xử lý nhiều lần để đảm bảo ảnh đầu ra phù hợp với prompt

Text2image: Tái tạo ảnh

- Latent image:
 - Tensor cuối sau N bước giảm nhiễu, chứa thông tin hình ảnh ở dạng nén, chưa phải ảnh thật
- Decoder:
 - Chuyển đổi tensor thành ảnh thực tế trong không gian pixel
 - Thông tin về màu sắc, ánh sáng, và cấu trúc hình ảnh được khôi phục từ dữ liệu trong latent space
- Kết quả:
 - Ảnh tạo ra phù hợp với nội dung của prompt đầu vào



Quá trình chuyển đổi từ latent image sang ảnh trong không gian pixel



Ví dụ ảnh được sinh ra

Image2image

- Quy trình
 - Ảnh đầu vào → Encoder → Tensor (latent space)
 - Thêm nhiễu vào tensor để điều chỉnh độ khác biệt so với ảnh gốc đầu vào
- Điều chỉnh nhiễu thêm vào
 - Kiểm soát mức độ nhiễu, xác định mức độ khác biệt giữa ảnh đầu vào và ảnh đầu ra
 - Nhiều nhẹ: ảnh đầu ra chỉ có thay đổi nhỏ với ảnh đầu vào
 - Nhiều mạnh: ảnh đầu ra khác biệt so với ảnh đầu vào

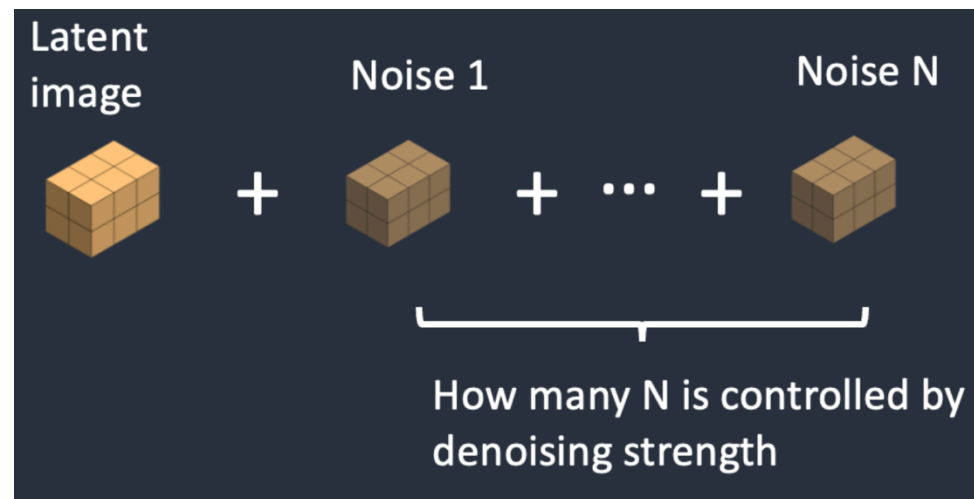
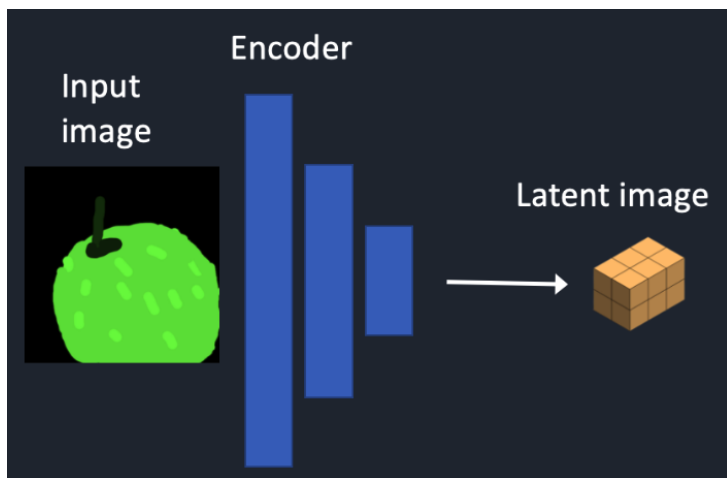
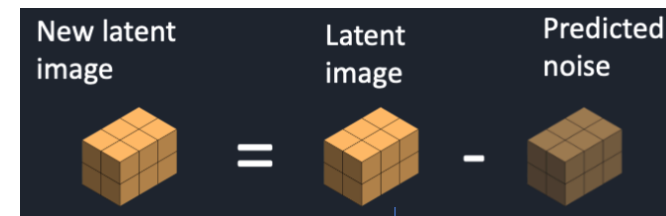
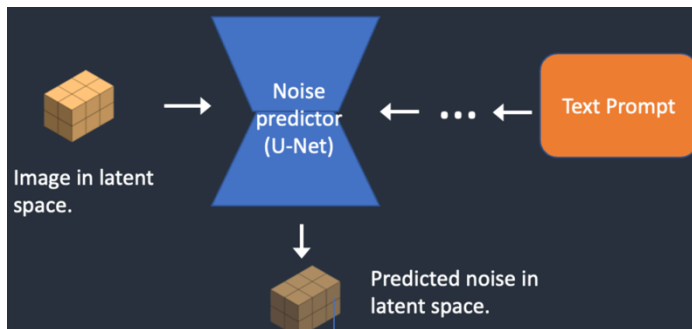


Image2image – Sử dụng Noise Predictor

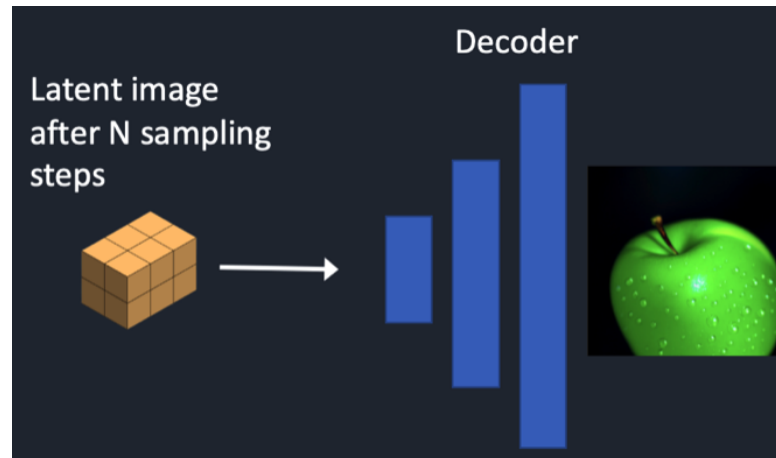
- Đầu vào:
 - Latent image chứa thông tin ảnh ban đầu sau khi thêm nhiễu
 - Text prompt hướng dẫn quá trình giảm nhiễu
- Noise predictor
 - Dự đoán nhiễu cần loại bỏ trong tensor hiện tại
- Quy trình
 - Dự đoán nhiễu → cập nhật tensor → lặp lại N lần nhất định → tensor (latent image) đầu ra



x 15-20 times

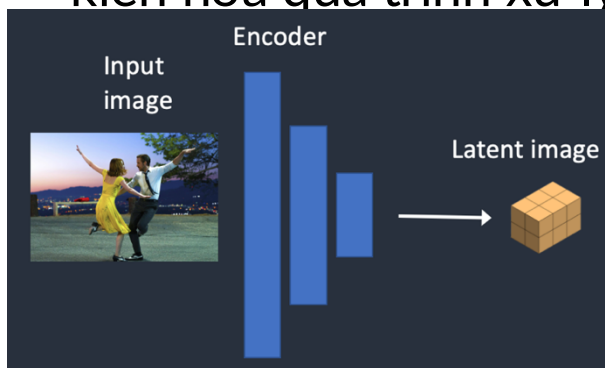
Image2image – Sử dụng Decoder tạo ảnh mới

- Latent image cuối cùng
 - Đầu ra quá trình giảm nhiễu thông qua N bước lặp
 - Chứa các đặc điểm để tạo ra hình ảnh cuối cùng
- Decoder
 - Giải mã từ latent space (tensor) sang pixel space (ảnh thực tế)
 - Thông tin về màu sắc, chi tiết bề mặt, ánh sáng và cấu trúc hình ảnh được tái tạo
- Ảnh đầu ra
 - Hình ảnh cuối cùng được tạo ra từ ảnh đầu vào và thông tin trong text prompt

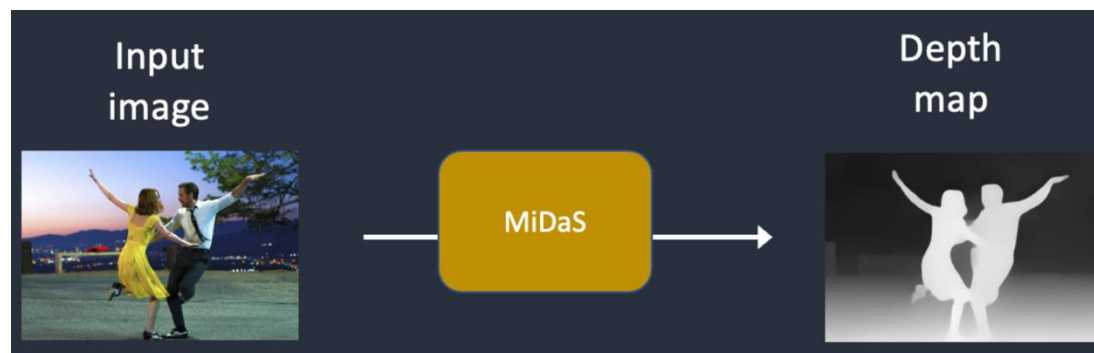


Depth2image

- Mục đích: tạo hình ảnh dựa trên thông tin độ sâu (depth) và các đặc tính của hình ảnh gốc
- Quy trình:
 - Dùng Encoder để chuyển hình ảnh đầu vào sang không gian ẩn (latent space)
 - Hình gốc được mã hóa thành tensor trong latent space giúp giảm kích thước và tập trung vào các đặc trưng quan trọng
 - Dùng MiDaS để trích xuất thông tin depth
 - MiDaS là mô hình chuyên dùng để tạo bản đồ độ sâu (depth map) từ hình ảnh đầu vào
 - Depth map cung cấp thông tin về khoảng cách về cấu trúc không gian, hỗ trợ điều kiện hóa quá trình xử lý



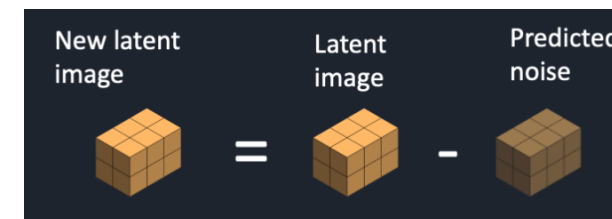
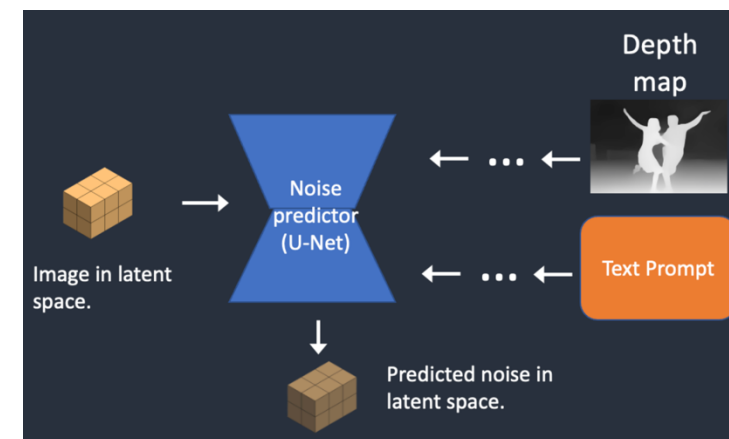
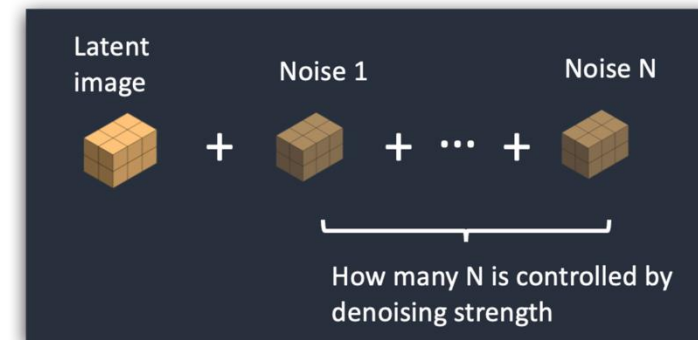
Mã hóa hình ảnh đầu vào
thành latent image



Sử dụng MiDaS để tạo
bản đồ độ sâu

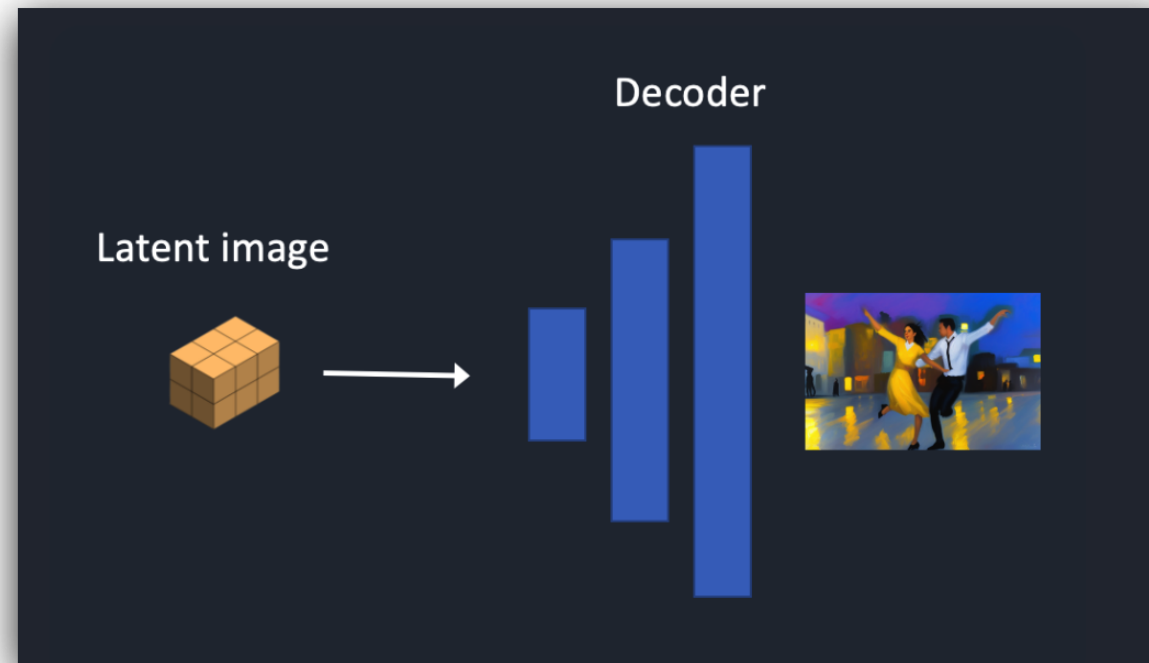
Depth2image

- Quy trình (tiếp)
 - Thêm nhiễu vào latent image
 - Latent image được cộng thêm nhiễu qua N bước
 - Tạo ra sự thay đổi cho latent image, chuẩn bị cho giai đoạn khử nhiễu
 - Noise predictor:
 - Đầu vào
 - Latent image đã thêm nhiễu
 - Depth map tính bởi MiDaS cung cấp thông tin về cấu trúc và khoảng cách
 - Text prompt để hướng dẫn nội dung ảnh mong muốn
 - Đầu ra: nhiễu cần loại bỏ
 - Cập nhật latent image
 - Latent image mới = latent image cũ - nhiễu tính từ noise predictor
 - Lặp lại quá trình tính nhiễu từ noise predictor và cập nhật latent image N lần



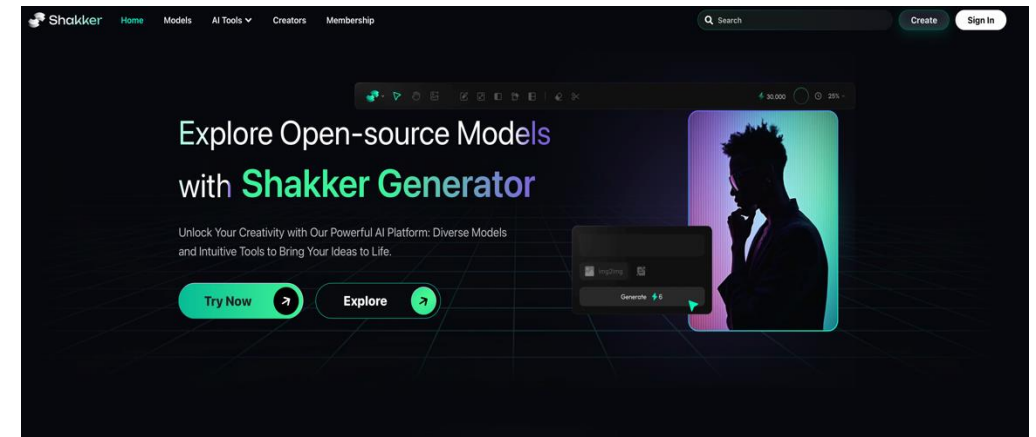
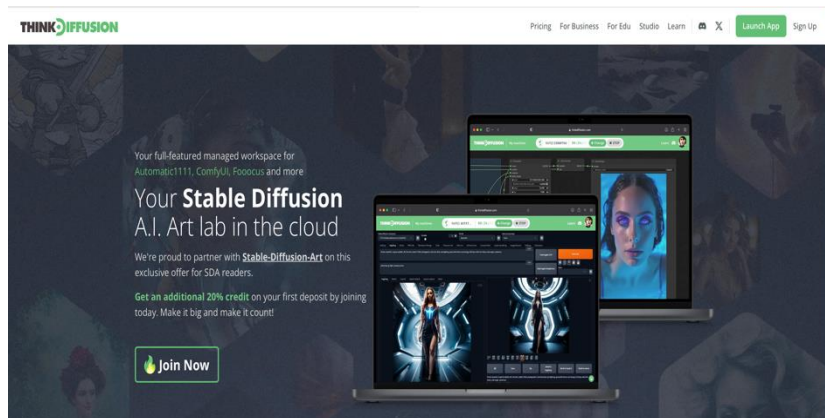
Depth2image

- Quy trình (tiếp)
 - Sử dụng decoder để chuyển latent image tính được sau N bước trở thành ảnh đầu ra trong không gian pixel



Stable diffusion WebUI (AUTOMATIC 1111)

- Stable-diffusion không có giao diện người dùng, viết code → cần có GUI
- Một trong những GUI đầu tiên: AUTOMATIC1111
 - Sử dụng: thương mại hóa: <https://www.thinkdiffusion.com> / <https://www.shakker.ai>



- Google Colab/ Cài đặt trên máy tính

Sử dụng AUTOMATIC1111 (1)

- Text-to-image tab: khi khởi động

Stable Diffusion checkpoint
v1-5-pruned-emaonly.ckpt [cc6cb27103] 

txt2img img2img Extras PNG Info Checkpoint Merger Train Settings Extensions

Prompt (press Ctrl+Enter or Alt+Enter to generate)

Negative prompt (press Ctrl+Enter or Alt+Enter to generate)

Generate

← 🗑️ 📄 📁 💾

Styles 

Sampling method DPM++ 2M Karras Sampling steps 25

Width 512 Height 512 



Sử dụng AUTOMATIC1111 (2)

- Các cấu hình quan trọng
 - Stable Diffusion Checkpoint: Lựa chọn mô hình muốn sử dụng, ví dụ v1.5 base model
 - Nhiều models khác nhau, cần chú ý lựa chọn
 - Prompt: Đưa prompt của mình vào
 - Width và height: Kích cỡ ảnh đầu ra
 - Batch size: số lượng ảnh sinh ra mỗi lần chạy
 - Ấn generate để sinh ảnh

Stable Diffusion checkpoint
v1-5-pruned-emaonly.ckpt [cc6cb27103]

txt2img | img2img | Extras | PNG Info | Checkpoint Merger | Train | Settings

Prompt (press Ctrl+Enter or Alt+Enter to generate)

Negative prompt (press Ctrl+Enter or Alt+Enter to generate)

Sampling method
DPM++ 2M Karras

Sampling steps
25

Width
512

Height
512

CFG Scale

Batch count
1

Batch size
4

Seed
-1

Extra

Restore faces | Tiling | Hires. fix

Script
None

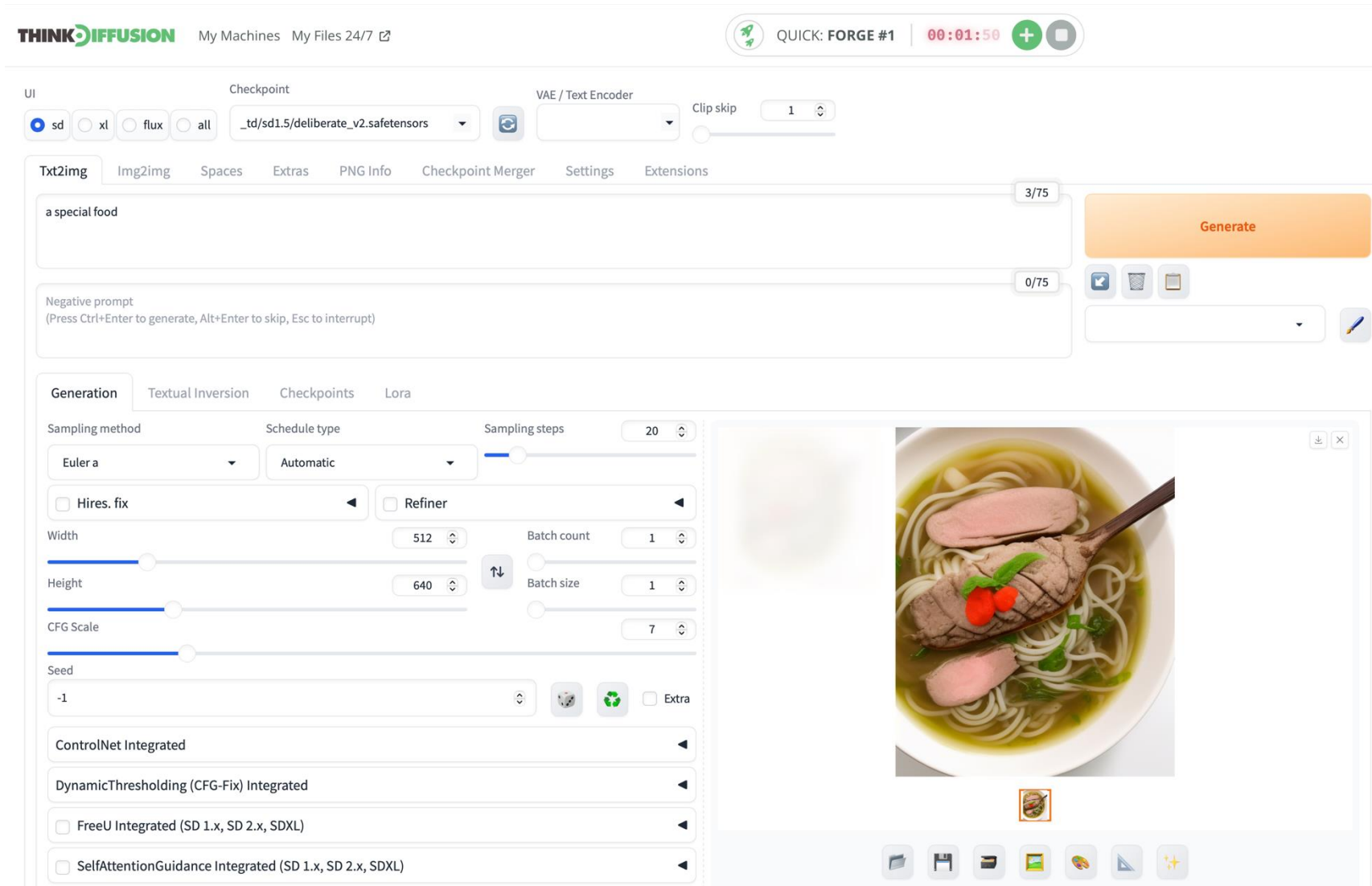
Sử dụng AUTOMATIC1111 (3)

- Các cấu hình khác
 - Negative prompt: nội dung không muốn hiển thị trong ảnh
 - Sampling method: thuật toán cho quá trình khử nhiễu
 - Sampling steps: số bước trong quá trình khử nhiễu. Càng nhiều càng tốt hơn nhưng lâu hơn
 - Batch count: số lần chạy quy trình (pipeline) tạo ảnh
 - Batch size: số lượng ảnh sinh ra mỗi pipeline → số lượng ảnh: batch count × batch size
 - CFG scale: Kiểm soát mức độ mà mô hình nên tuân theo yêu cầu của bạn, càng cao thì càng tuân theo
 - 1: bỏ qua
 - 30: tuân nghiêm ngặt
 - Seed: tạo tensor ngẫu nhiên trong latent space, thường đặt là -1

The screenshot shows the AUTOMATIC1111 web interface with the 'txt2img' tab selected. The 'Stable Diffusion checkpoint' dropdown is set to 'v1-5-pruned-emaonly.ckpt [cc6cb27103]'. The 'Prompt' and 'Negative prompt' fields are empty. The 'Sampling method' is set to 'DPM++ 2M Karras'. The 'Sampling steps' slider is at 25. The 'Width' and 'Height' sliders are both at 512. The 'CFG Scale' slider is at 7. The 'Batch count' is 1 and the 'Batch size' is 4. The 'Seed' is set to -1. There are buttons for 'Restore faces', 'Tiling', and 'Hires. fix'. The 'Script' dropdown is set to 'None'.

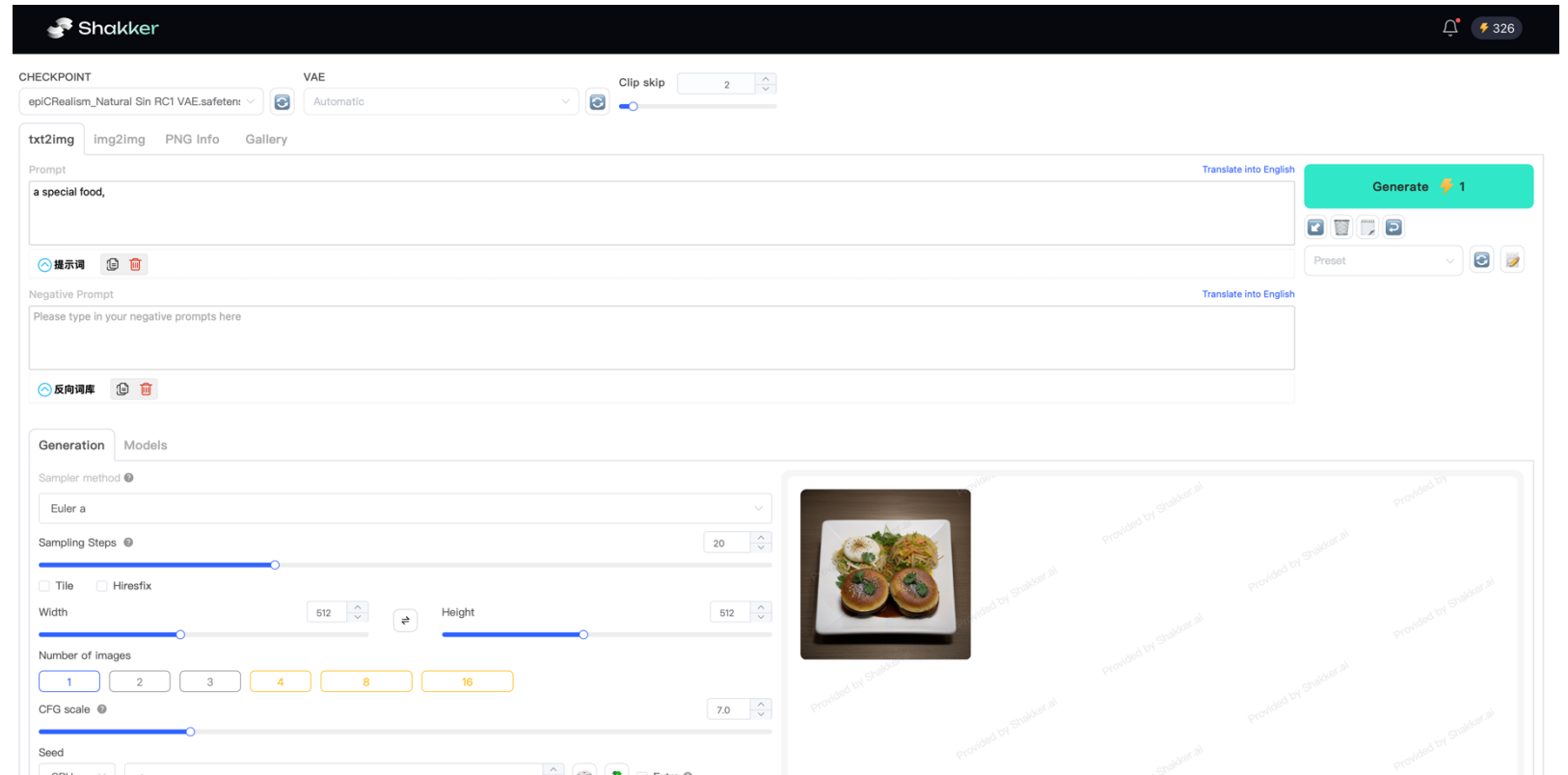
Sử dụng AUTOMATIC 1111 (4)

- Sinh ảnh
- Có thể download ảnh



Sử dụng AUTOMATIC 1111 (5)

- Sinh ảnh

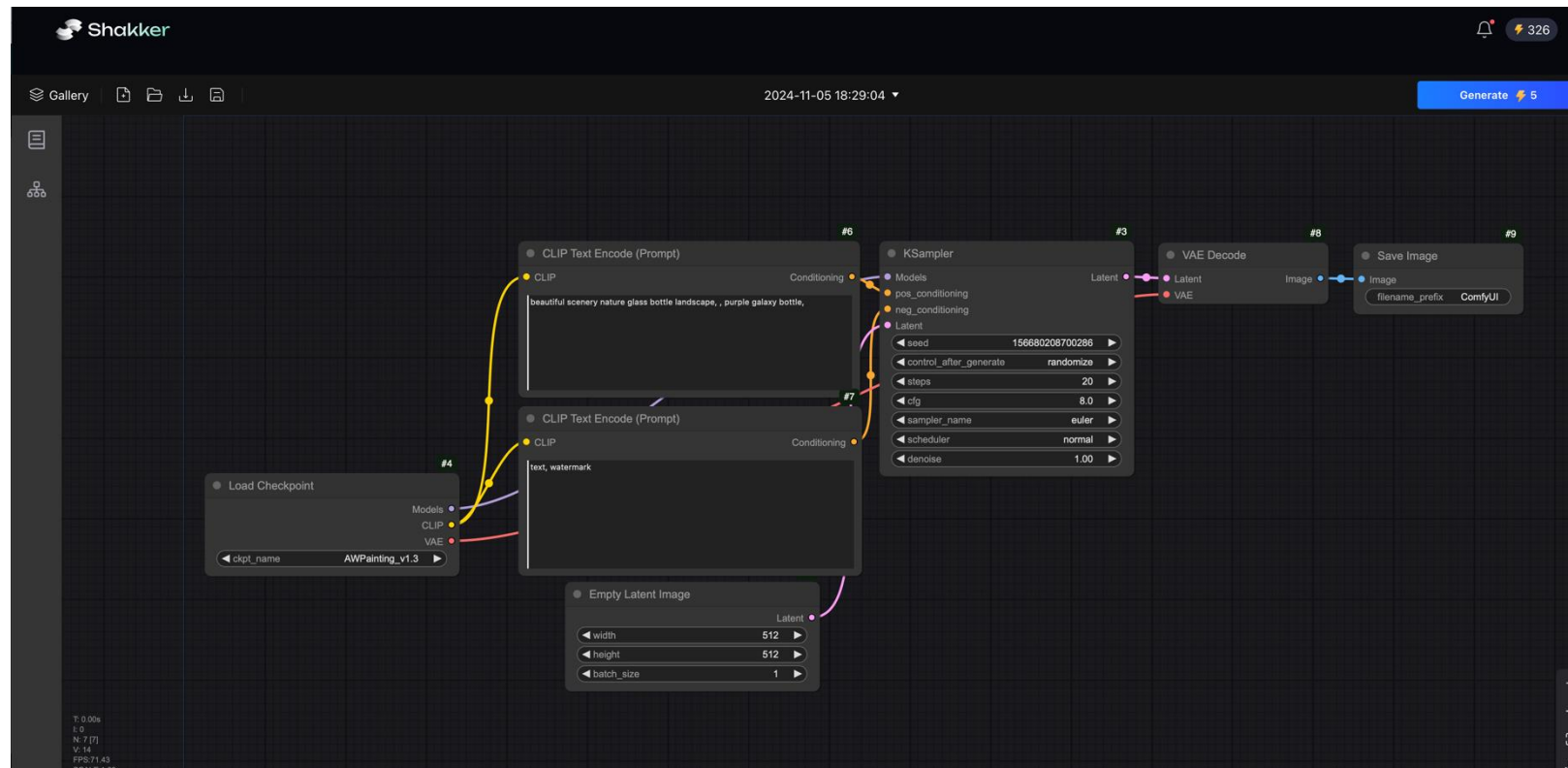


Sử dụng ComfyUI

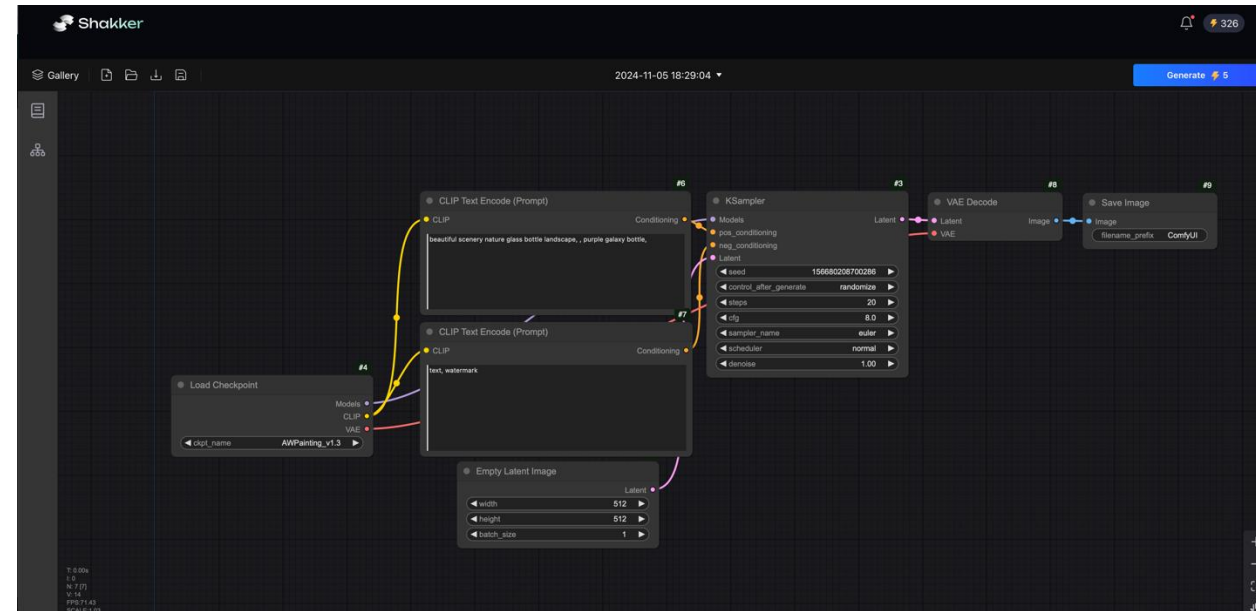
- Node-based GUI cho stable diffusion
- Sử dụng:
 - Thương mại hóa: Think diffusion/ Shakker AI
 - ComfyUI notebook: Google colab/ local
- So sánh với AUTOMATIC1111
 - Ưu điểm:
 - Nhẹ hơn
 - Cấu hình linh hoạt hơn
 - Dễ quan sát được workflow
 - Dễ chia sẻ : mỗi file được chia sẻ là 1 workflow dễ dàng chạy lại
 - Nhược điểm:
 - Giao diện không nhất quán: các workflow khác nhau có thể đặt các node khác nhau
 - Quá chi tiết: xem các kết nối bên trong

Sử dụng ComfyUI (2)

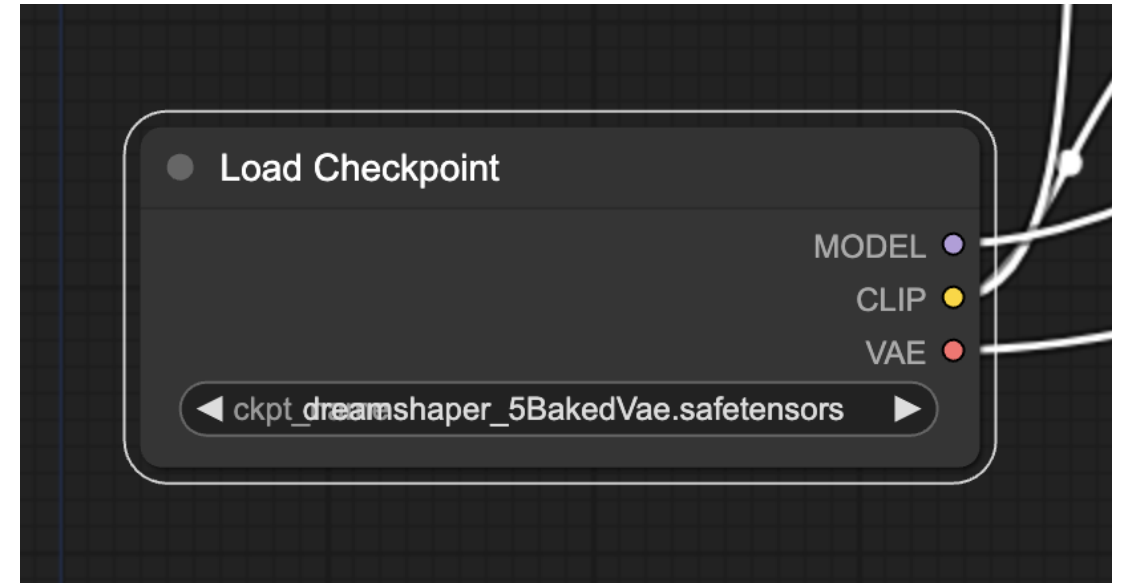
- Điều khiển cơ bản
 - Chuột hoặc touchpad để phóng to thu nhỏ
 - Kéo và giữ các điểm input/output để tạo ra kết nối (cùng loại)
 - Giữ và kéo chuột trái để di chuyển không gian làm việc



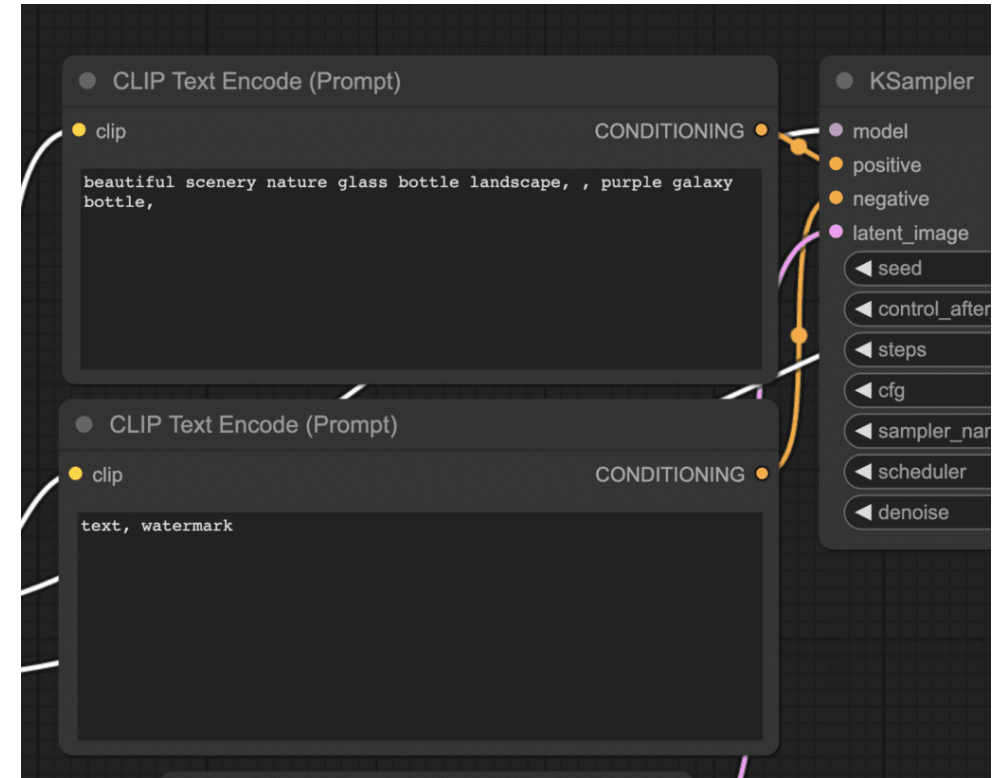
- Node:
 - Khối hình chữ nhật (ví dụ “load checkpoint”)
 - Mỗi node thực hiện được đoạn code (tương tự các hàm)
 - Cần ba thành phần
 - Input: các đoạn text và dấu chấm bên trái (có các kết nối đi vào)
 - Output: các đoạn text và dấu chấm bên phải (có các kết nối đi ra)
 - Tham số (parameter): các trường nằm giữa khối hình chữ nhật
- Edge: Các kết nối giữa đầu ra và đầu vào của các khối



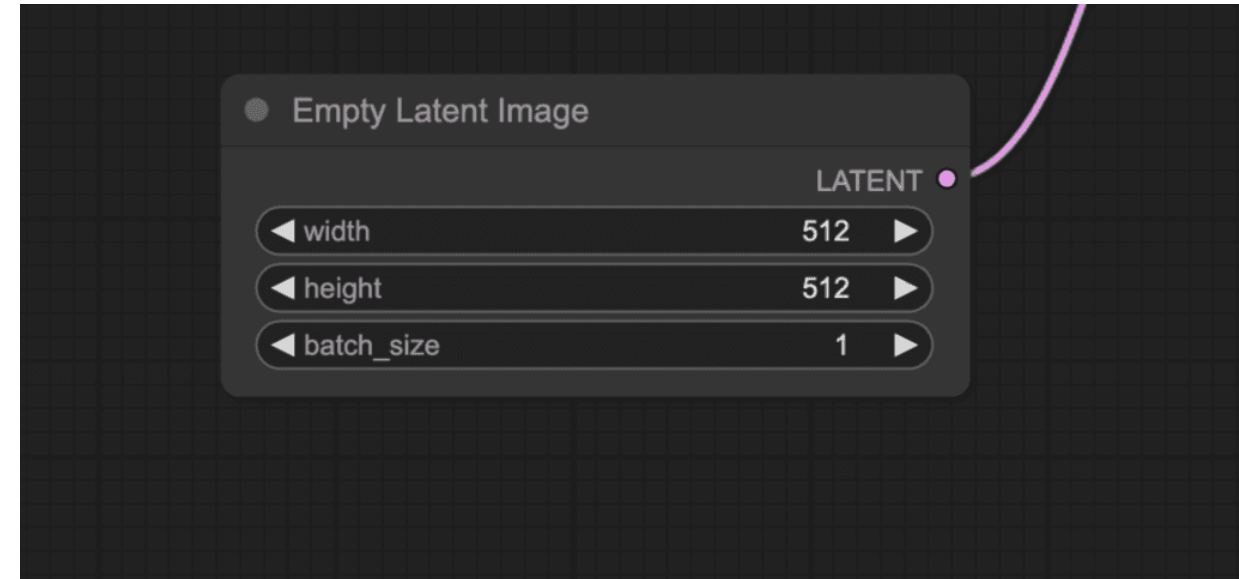
- Lựa chọn model
 - Lựa chọn stable diffusion model ở Load checkpoint node
 - Ba thành phần
 - MODEL:
 - Mô hình dự đoán nhiễu trong không gian ẩn (latent space)
 - Nối với Sampler: xử lý quá trình khuếch tán ngược
 - CLIP:
 - Mô hình ngôn ngữ xử lý prompt
 - Nối với prompt (CLIP Text Encode) để xử lý
 - VAE:
 - Chuyển đổi giữa hình ảnh và không gian ẩn
 - Trong text-to-image: chỉ sử dụng ở bước cuối để chuyển hình ảnh từ không gian ẩn sang không gian điểm ảnh (pixel space): Chỉ sử dụng phần giải mã



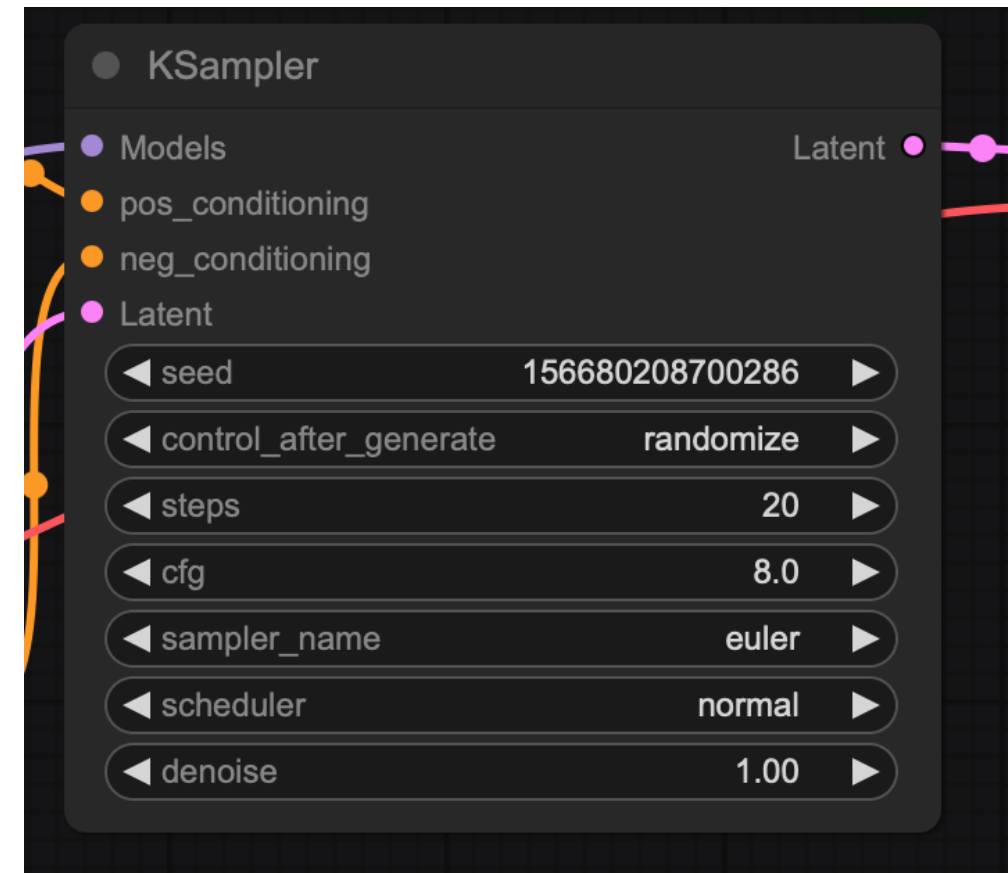
- CLIP Text Encode: Nhập prompt và negative prompt
 - Text được encode sang token và embeddings
 - Sử dụng mô hình ngôn ngữ CLIP của OpenAI
 - Prompt và negative prompt được xác định bởi CONDITIONING



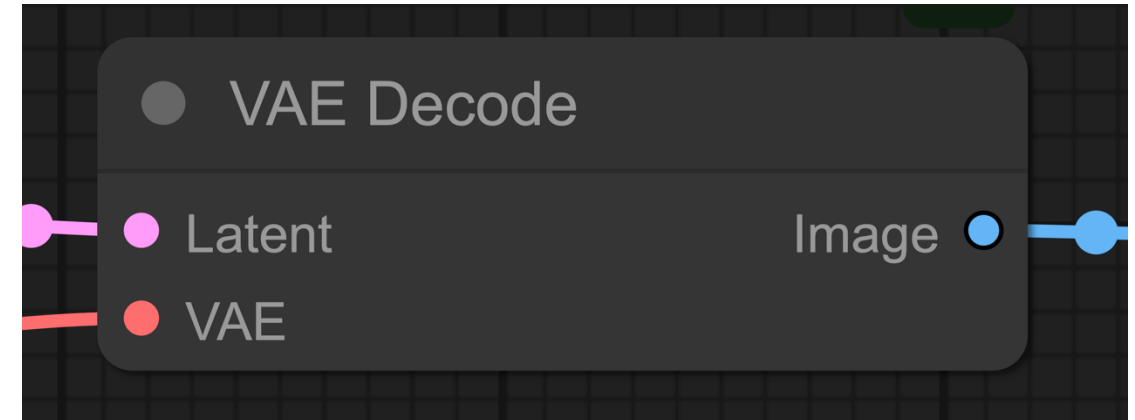
- Empty latent space
 - Text-to-image: bắt đầu từ latent space
 - Kích thước của hình ảnh trong không gian ẩn (kích thước ảnh thu được): width/height
 - Batch size: số ảnh tạo ra trong mỗi lần chạy



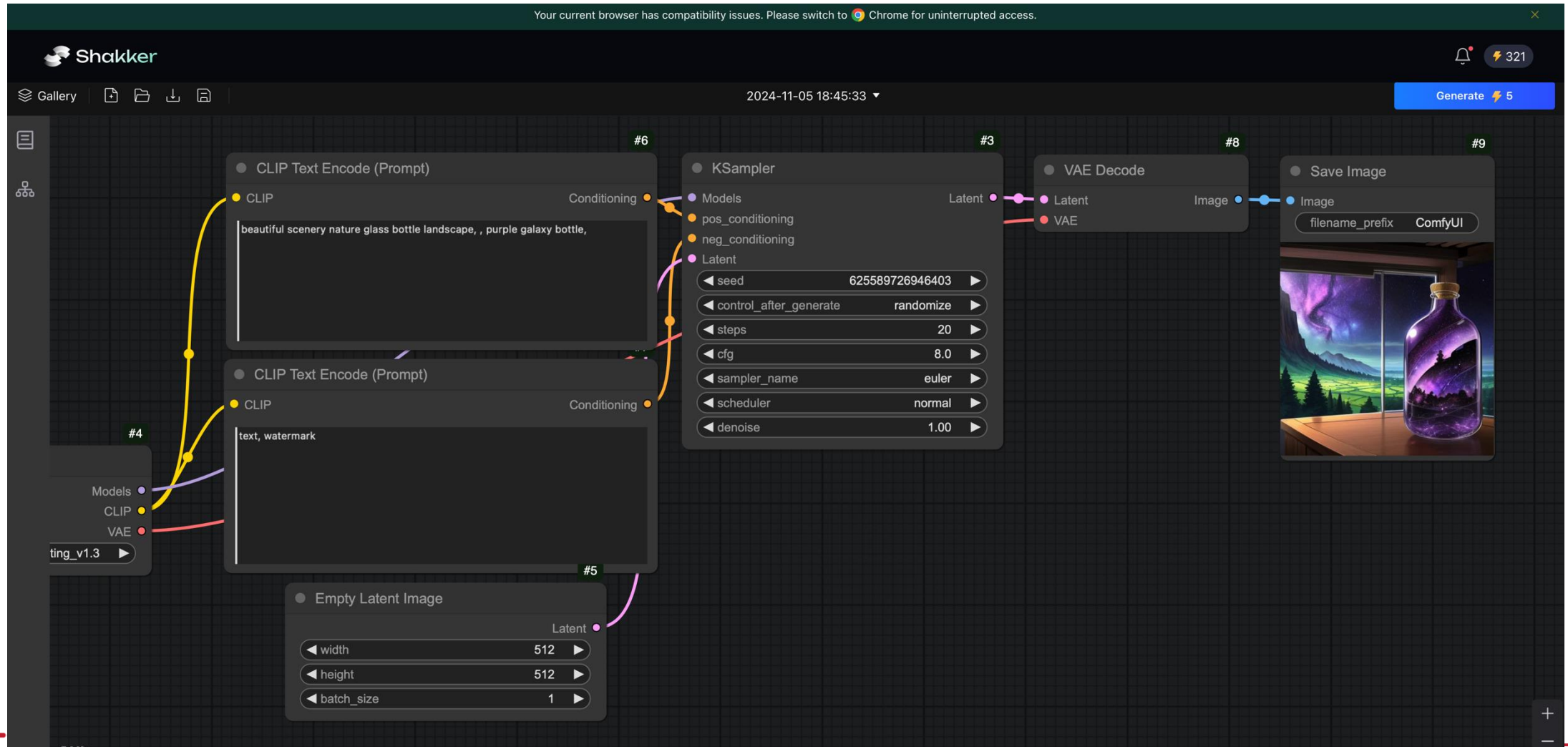
- Ksampler
 - Thành phần quan trọng để sinh hình ảnh, khử nhiễu để tạo hình ảnh phù hợp với prompt
 - Các tham số quan trọng
 - seed: Giá trị ngẫu nhiên điều khiển việc sinh latent space
 - control_after_generate: cách thay đổi seed sau mỗi lần tạo: nhận giá trị ngẫu nhiên/ tăng 1/ giảm 1/ hay giữ nguyên
 - step: số bước thực hiện
 - cfg: kiểm tra độ mạnh của prompt
 - sampler_name: thuật toán khử nhiễu
 - scheduler: Kiểm soát mức độ nhiễu thay đổi ở mỗi bước.
 - denoise: Kiểm soát lượng nhiễu ban đầu cần loại bỏ trong quá trình khử nhiễu. 1 nghĩa là loại bỏ hoàn toàn.



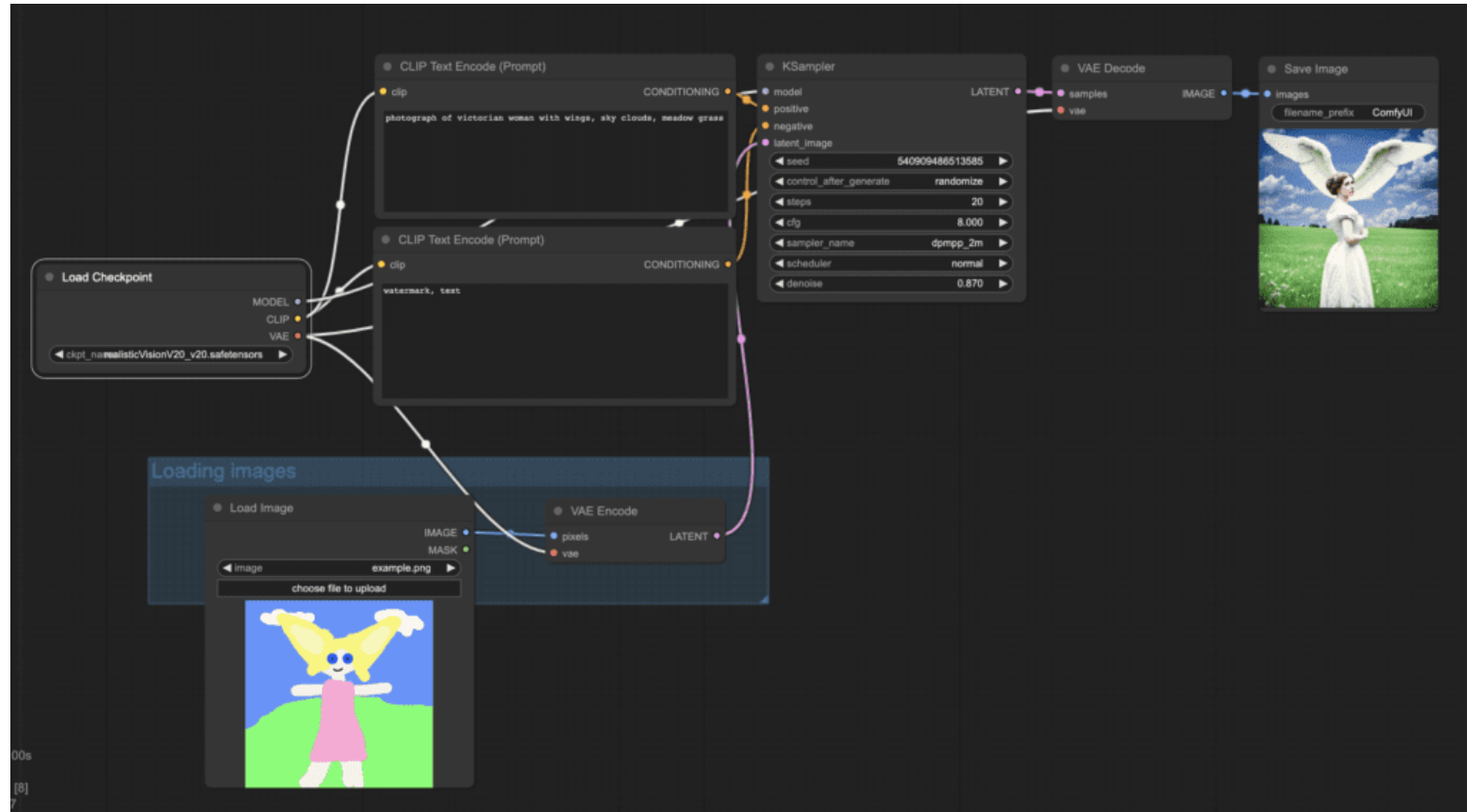
- VAE Decode
 - Tạo ra hình ảnh từ không gian ẩn
 - Latent: Hình ảnh trong latent space chứa thông tin được nén của hình ảnh thực, nhận từ Ksampler
 - VAE: Dùng để giải mã để chuyển từ latent space sang pixel space
 - Image: Ảnh đầu ra thu được



- Sinh ảnh dùng shakker AI



- Thêm ảnh đầu vào kèm prompt để sinh ra ảnh theo ý muốn



Câu hỏi

1. Mô hình khuếch tán hoạt động như thế nào?
2. Điều gì làm cho Stable Diffusion khác biệt so với khuếch tán thông thường?
3. (Bổ sung) Sự khác biệt giữa Stable Diffusion và mô hình SDXL là gì?
4. (Bổ sung) Làm thế nào văn bản hình ảnh được biểu diễn trong cùng một miền sử dụng CLIP